DAY 2 | POST-READ

Python Basics, Functions and Files: Recap and Beyond

What we covered today, plus self-study topics and ideas to stretch your understanding.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Part A: Summary of today

Topic	What to remember
Variables and types	int, float, str, bool. = stores, == compares. Check with type().
Type casting	input() returns text. Use int(), float(), str() to convert.
Conditions	if, elif, else. Only the first True branch runs. Colon and indentation required.
Loops	for over a sequence, while until a condition changes. break and continue.
Functions	def name(params): ... return value. No return means None.
Collections	list ordered, tuple fixed, set unique, dict key value. get() for safe lookups.
Nested data	A list of dicts is a table of records and the shape of JSON.
Files	with open(path, mode). csv.DictReader / DictWriter. json.load / json.dump.

Part B: Self-study topics

1. List comprehensions

Build a new list in one readable line: [expression for item in items if condition].

```
prices = [120, 450, 80, 990]
with_tax = [round(p * 1.18, 2) for p in prices]
cheap = [p for p in prices if p < 200]   # [120, 80]
```

2. Dictionary and set comprehensions

```
names = ["ana", "ken", "ana"]
lengths = {n: len(n) for n in names}    # {'ana': 3, 'ken': 3}
unique = {n.upper() for n in names}     # {'ANA', 'KEN'}
```

3. lambda, map and filter

A lambda is a tiny one-line function. map applies a function to every item; filter keeps items where the function returns True. Comprehensions often read better, but you will see both.

```
square = lambda x: x * x
print(list(map(square, [1, 2, 3])))            # [1, 4, 9]
print(list(filter(lambda p: p < 200, prices)))  # [120, 80]
top = sorted(tickets, key=lambda t: t["priority"])
```

Part C: Going beyond today

1. Mutable default arguments: a classic trap

Never use a list or dict as a default value. It is created once and shared between calls. Use None.

```
def add_item(item, basket=None):
    if basket is None:
        basket = []
    basket.append(item)
    return basket
```

2. Safe casting with try and except

If a user types "abc" where a number is expected, int() raises a ValueError. Catch it and ask again. Exceptions are covered fully on Day 3.

3. pathlib and type hints

Path("data") / "marks.csv" builds paths safely on any operating system. Type hints such as def average(marks: list[float]) -> float do not change how code runs, but editors and FastAPI (Day 5) use them heavily.

Part D: Check yourself

- What is the difference between = and ==?
- What is the difference between a parameter and an argument?
- When would you choose a tuple over a list, and a dict over a list?
- Why does json.dump keep numbers as numbers but CSV does not?
- What does the with statement do for you?

Part E: Practice ideas for tonight

- Print 1 to 30, with "Fizz" for multiples of 3 and "Buzz" for multiples of 5.
- Read a CSV of expenses and print the total per category.
- Add a "close ticket" function to your SmartDesk ticket logger.

Coming up on Day 3: classes and objects, exceptions, dependency hygiene, and your first afternoon with OpenAI Codex in VS Code.

Otsukaresama deshita (お疲れ様でした): thank you for your hard work today